

# 09 - Introducción a Javascript

## ¿Qué es JavaScript?

JavaScript (JS) es un lenguaje de programación que se utiliza para crear páginas web interactivas. A diferencia de HTML y CSS, que solo se encargan de la estructura y el estilo de una página, JavaScript puede hacer que las páginas respondan a las acciones del usuario, realizar cálculos, manejar eventos, y mucho más.

Es un lenguaje muy popular porque puede ejecutarse en prácticamente cualquier navegador web, sin necesidad de instalar nada adicional. Además, JavaScript se usa tanto en el front-end (lo que ve el usuario) como en el back-end (la parte del servidor).

## Sintaxis básica de JavaScript

Al igual que otros lenguajes de programación, JavaScript tiene reglas y estructuras que debemos seguir. Algunas cosas básicas que debes conocer:

**Instrucciones y declaraciones:** En JavaScript, cada acción que deseas que el programa realice se llama instrucción y generalmente termina con un punto y coma ;.

```
console.log("Hola, mundo!"); // Imprime "Hola, mundo!" en la consola
```

**Comentarios:** Puedes agregar comentarios en tu código para explicar lo que estás haciendo. Hay dos tipos:

```
Comentarios de una línea: // Este es un comentario de una línea  
Comentarios de varias líneas:
```

```
/*  
Este es un comentario  
de varias líneas  
*/
```

## ES6: Una versión mejorada de JavaScript

ES6 (o ECMAScript 2015) fue una actualización importante del lenguaje que introdujo muchas características nuevas. Verás que muchas de las cosas modernas que hacemos en JavaScript están relacionadas con ES6. Algunas de las características más útiles incluyen las nuevas palabras clave para declarar variables: `let` y `const`, las funciones de flecha, y más.

### Declaración de variables: `var`, `let` y `const`

Una de las primeras cosas que aprenderás en JavaScript es cómo declarar variables, es decir, nombres que usas para almacenar datos. Las tres formas principales de hacerlo son usando `var`, `let` o `const`.

#### `var` (antes de ES6)

Antes de ES6, la única forma de declarar variables era usando `var`. Sin embargo, `var` tiene algunos problemas, ya que las variables declaradas con `var` tienen alcance global o de función, lo que puede causar errores si no se tiene cuidado.

```
var nombre = "Juan";  
var edad = 25;  
console.log(nombre); // Imprime "Juan"
```

#### `let` (ES6)

Con ES6 llegó `let`, que es más seguro que `var` porque las variables tienen alcance de bloque. Esto significa que la variable solo existe dentro del bloque `{ }` en el que fue definida.

```
let nombre = "Ana";
if (true) {
  let edad = 30;
  console.log(edad); // Imprime 30
}
console.log(edad); // Error: La variable edad no existe fuera del bloque
```

### **const** (ES6)

`const` se usa para declarar constantes, es decir, valores que no cambian una vez asignados. Si intentas cambiar una variable declarada con `const`, JavaScript te dará un error.

```
const pi = 3.14;
console.log(pi); // Imprime 3.14
pi = 3.15; // Error: no puedes reasignar una constante
```

### ¿Cuándo usar `let` y `const`?

- Usa `let` cuando necesites cambiar el valor de una variable más adelante.
- Usa `const` cuando el valor no deba cambiar.

## Tipos de datos en JavaScript

JavaScript tiene varios tipos de datos. Estos definen qué tipo de valores puedes almacenar en tus variables. Vamos a ver los más comunes:

### 1. Números (Number)

En JavaScript, los números se usan tanto para enteros como para decimales.

```
let entero = 42;  
let decimal = 3.14;
```

## 2. Cadenas de texto (String)

Las cadenas son texto, y pueden declararse con comillas simples (') o dobles ("). También puedes usar comillas invertidas (`) para crear plantillas de cadenas.

```
let saludo = "Hola";  
let nombre = 'Ana';  
let mensaje = `¡Bienvenido, ${nombre}!`; // Plantilla de cadena
```

## 3. Booleanos (Boolean)

Los booleanos solo tienen dos valores: true (verdadero) o false (falso). Son útiles para tomar decisiones en tu código.

```
let esAdulto = true;  
let esEstudiante = false;
```

## 4. Arreglos (Array)

Un arreglo es una colección de elementos, que pueden ser de cualquier tipo. Se definen con corchetes [].

```
let numeros = [1, 2, 3, 4];  
let mezclado = [true, 42, "Hola"];
```

## 5. Objetos (Object)

Un objeto es una estructura que puede almacenar datos en pares clave-valor. Los objetos son muy importantes en JavaScript.

```
let persona = {  
  nombre: "Juan",  
  edad: 30,  
  esEstudiante: false  
};
```

## 6. Null y Undefined

**null:** Es un valor que representa "nada". Lo asignas a una variable cuando quieres dejar claro que no tiene valor.

**undefined:** Es el valor que obtiene una variable cuando no se le ha asignado ningún valor.

```
let vacio = null;  
let indefinido;
```

## Operadores en JavaScript

Los operadores son símbolos que nos permiten realizar operaciones con los valores de nuestras variables. Aquí tienes algunos de los más comunes:

### Operadores aritméticos

Estos operadores permiten realizar cálculos matemáticos básicos.

```
let suma = 5 + 3;    // 8  
let resta = 10 - 6;  // 4  
let multiplicacion = 4 * 2; // 8
```

```
let division = 8 / 2; // 4
```

## Operadores de comparación

Nos permiten comparar dos valores. Por ejemplo:

```
let esIgual = 5 == "5"; // true (comparación flexible)
let esEstrictamenteIgual = 5 === "5"; // false (comparación estricta)
let esMayor = 10 > 5; // true
```

## Operadores lógicos

Son útiles cuando trabajamos con valores booleanos.

```
let esVerdadero = true && false; // false (AND)
let esUnoVerdadero = true || false; // true (OR)
let negacion = !true; // false (NOT)
```

Ejemplo de código básico en JavaScript

Vamos a crear un pequeño ejemplo usando algunas de las cosas que acabamos de ver:

```
let nombre = "Carlos";
const pi = 3.14;
let esEstudiante = true;

if (esEstudiante) {
  console.log(`${nombre} es estudiante y conoce el valor de pi: ${pi}`);
} else {
```

```
console.log(`${nombre} no es estudiante.`);  
}
```

¿Qué pasa aquí?

Declaramos una variable nombre y una constante pi.

Usamos un if para verificar si la variable esEstudiante es true. Si lo es, imprimimos un mensaje en la consola.

## Introducción a console.log, alert y prompt en JavaScript

Cuando empezamos a escribir código en JavaScript, necesitamos maneras de interactuar con los usuarios o ver lo que está sucediendo en nuestro programa. Para eso, JavaScript nos proporciona algunas herramientas simples como console.log, alert y prompt. Cada una de ellas tiene un propósito diferente y son fáciles de usar.

**console.log** - Mostrar información en la consola

El método console.log se usa para imprimir mensajes o valores en la consola del navegador. Es muy útil para depurar tu código, ver valores de variables o simplemente mostrar información.

Sintaxis

```
console.log(mensaje);
```

Puedes mostrar cualquier cosa dentro de los paréntesis: texto, números, resultados de cálculos o valores de variables.

Ejemplo

```
console.log("Hola, mundo"); // Imprime "Hola, mundo"  
console.log(10 + 5); // Imprime 15  
let nombre = "María";  
console.log(nombre); // Imprime "María"
```

Cuando abres la consola en el navegador (con F12 en muchos navegadores), puedes ver estos mensajes. Es una herramienta clave para los programadores, ya que te permite ver lo que está ocurriendo en el código en tiempo real.

### **alert** - Mostrar mensajes emergentes

El método `alert` genera una pequeña ventana emergente en el navegador con un mensaje que quieras mostrar. Es útil si deseas alertar al usuario sobre algo importante, pero no deberías abusar de esta función, ya que interrumpe la interacción con la página.

Sintaxis

```
alert(mensaje);
```

Al igual que con `console.log`, puedes mostrar texto, números o el valor de una variable.

Ejemplo

```
alert("¡Bienvenido a mi sitio web!");  
let nombre = "Carlos";  
alert("Hola, " + nombre);
```

Al ejecutar este código, aparecerá una ventana emergente con el mensaje que hayas especificado. El usuario tiene que cerrar la alerta para continuar interactuando con la página.

### **prompt** - Solicitar información al usuario

El método `prompt` es muy interesante porque no solo muestra un mensaje, sino que también solicita al usuario que introduzca algún dato. Este dato se guarda como una cadena de texto (string) y puedes almacenarlo en una variable para usarlo más adelante.

## Sintaxis

```
let variable = prompt(mensaje);
```

El mensaje dentro del prompt le indica al usuario qué tipo de información necesitas. El usuario escribe su respuesta en la ventana emergente y esa respuesta se almacena en la variable que definiste.

### Ejemplo

```
let nombre = prompt("¿Cómo te llamas?");  
alert("Hola, " + nombre + "!");
```

Este código pedirá al usuario que introduzca su nombre, lo almacenará en la variable nombre, y luego mostrará un mensaje de saludo personalizado.

### Ejercicio práctico con console.log, alert y prompt

#### Ejercicio 1:

Crea un pequeño programa que le pida al usuario su edad y luego muestre un mensaje en la consola diciendo si es mayor de edad o no.

#### Código

```
let edad = prompt("¿Cuántos años tienes?");  
if (edad >= 18) {  
    console.log("Eres mayor de edad");  
    alert("Eres mayor de edad");  
} else {  
    console.log("Eres menor de edad");  
    alert("Eres menor de edad");  
}
```

El usuario ingresa su edad a través de un prompt.

Dependiendo de la edad ingresada, se mostrará un mensaje en la consola y una alerta indicando si es mayor o menor de edad.

## Conclusión

En esta introducción a JavaScript, aprendiste los fundamentos de la sintaxis, las diferencias entre `var`, `let` y `const`, y los tipos de datos principales. Además aprendiste que los métodos `console.log`, `alert` y `prompt` son herramientas básicas pero poderosas para interactuar con el usuario y depurar tu código en JavaScript.

En los próximos capítulos, nos sumergiremos más profundamente en la lógica de control, funciones, arrays, objetos, y más.

Este es solo el comienzo. ¡Prepárate para descubrir todo el potencial de este lenguaje y cómo te permite interactuar con las páginas web!